FINANCIAL RISK ANALYSIS

Blackrock — Customer Financial Behavior & Risk Analysis Report

800 transactions • 184 accounts • 180 customers • Jan 2023 – Jun 2024

Video Presentation Link: https://www.loom.com/share/6e7ec8f7ea974ce89e77f686d7c12a9c

Prepared by: Sagar | Tools: Python (pandas, numpy, matplotlib, seaborn)

What This Report Covers

01 Data Cleaning & Formatting

02 Descriptive Transactional Analysis

03 Customer Profile Building

04 Financial Risk Identification

05 Visualisation

06 Hypothesis Testing

Task 1 — Data Cleaning & Formatting

- Stripped special characters from TransactionAmount, AccountBalance, RiskScore & CreditRating, then coerced to numeric
- Parsed TransactionDate into proper datetime format
- Standardized AccountType & TransactionCategory to Title Case
- Checked for duplicate records

0	0
Missing values	Duplicate records
0	**800**
Invalid dates	Clean transactions

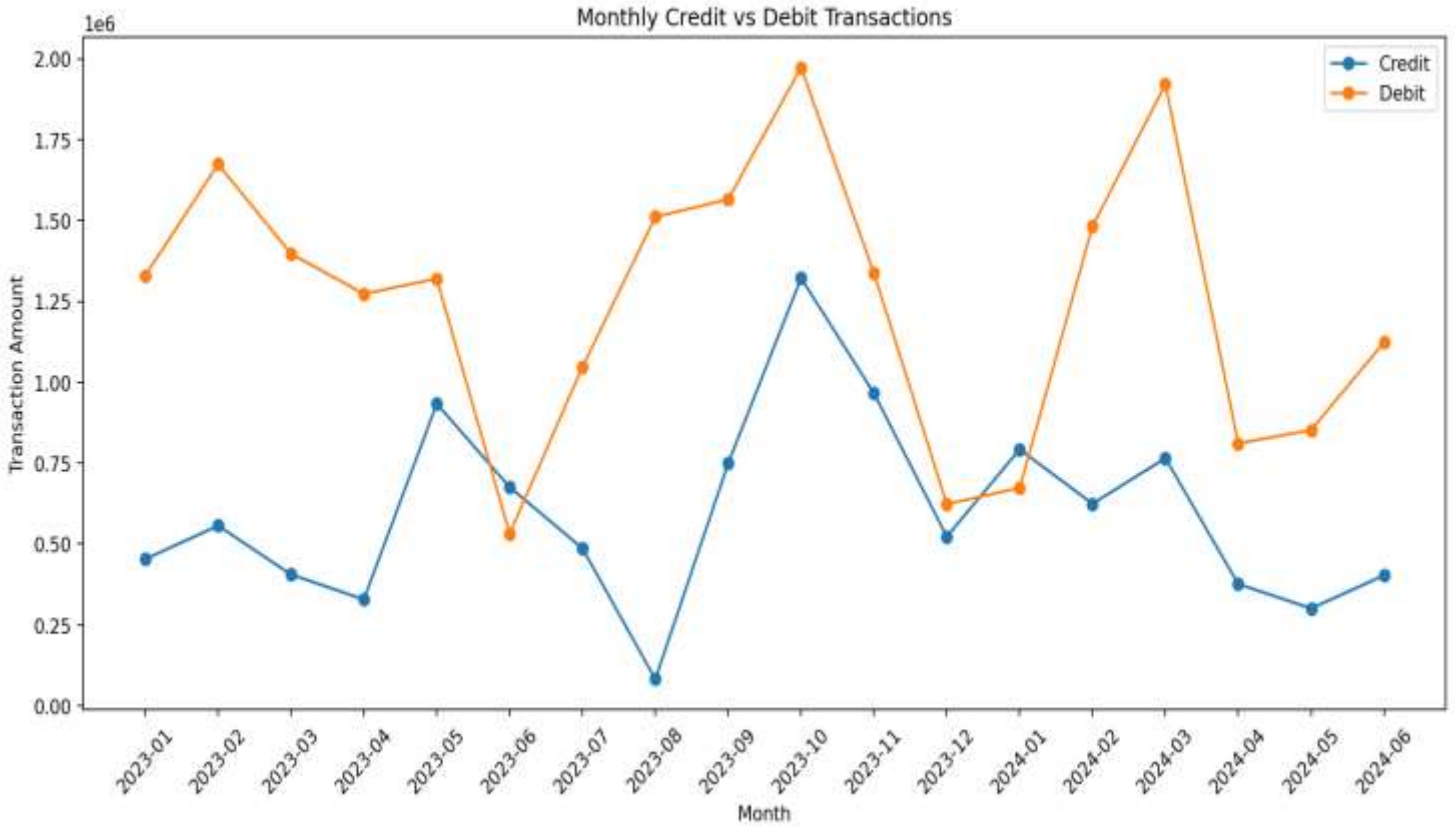
Cleaned dataset saved as blackrock_cleaned.csv

TASK 1 — Python Code

Data Cleaning & Formatting

```
#DATA CLEANING
print("\nOriginal Data:")
print(df.head())
print("\nData Types Before Cleaning:")
print(df.dtypes)
print("\nMissing Values:")
print(df.isnull().sum())
#Remove special characters from financial fields
financial_columns = ["TransactionAmount", "AccountBalance",
                     "RiskScore", "CreditRating"]
for col in financial_columns:
    df[col] = df[col].astype(str).str.replace(r"[^\d.-]", "", regex=True)
    df[col] = pd.to_numeric(df[col], errors="coerce")
# Convert TransactionDate into datetime
df["TransactionDate"] = pd.to_datetime(df["TransactionDate"], errors="coerce")
# Standardize text fields
df["AccountType"] = df["AccountType"].astype(str).str.strip().str.title()
df["TransactionCategory"] = df["TransactionCategory"].astype(str).str.strip().str.title()
df["TransactionType"] = df["TransactionType"].astype(str).str.strip().str.title()
# Remove duplicate records
duplicate_count = df.duplicated().sum()
df = df.drop_duplicates()
# Save cleaned dataset
df.to_csv("blackrock_cleaned.csv", index=False)
```

Task 2 — Credit vs. Debit Trend



NET TRANSACTION VOLUME

- Rs. 1.17 Cr

Debits exceed credits in nearly every month

Rs. 1.07 Cr
Total credit volume

Rs. 2.24 Cr
Total debit volume

Task 2 — Account Performance & Dormancy

TOP 3 · NET INFLOW

ACC88286	+Rs. 2,84,231
ACC21719	+Rs. 2,74,729
ACC51200	+Rs. 2,01,470

BOTTOM 3 · NET OUTFLOW

ACC45101	-Rs. 3,95,420
ACC35419	-Rs. 3,65,901
ACC97411	-Rs. 3,65,711

DORMANCY RUBRIC

A gap of ≥60 days between consecutive transactions on an account is flagged as dormant.

172

of 184 accounts (93%) had at least one dormancy episode

325 individual transaction gaps met the 60-day threshold across the dataset — dormancy is the norm, not the exception.

Task 2 — Account Performance & Dormancy

TOP 10 · NET INFLOW

```
Top 10 Accounts by Net Inflow:
AccountID
ACC88286    284230.600790
ACC21719    274729.185720
ACC51200    201469.732440
ACC28295    199450.692920
ACC88252    183519.248820
ACC83269    180850.726890
ACC88449    134213.978110
ACC32627    125993.714440
ACC31902    103237.587600
ACC45907     99707.422688
Name: TransactionAmount, dtype: float64
```

BOTTOM 10 · NET OUTFLOW

```
Bottom 10 Accounts by Net Inflow:
AccountID
ACC70460   -271229.485120
ACC81631   -272715.698320
ACC11837   -280003.838417
ACC53466   -314267.167030
ACC78581   -332353.939521
ACC24070   -338136.693810
ACC71938   -353666.209440
ACC97411   -365710.863533
ACC35419   -365901.203200
ACC45101   -395419.947830
Name: TransactionAmount, dtype: float64
```

TASK 2 — Python Code

Descriptive Transactional Analysis

```
# Monthly & yearly summaries
monthly_summary = df.groupby(df["TransactionDate"].dt.to_period("M"))["TransactionAmount"] \
    .agg(TotalTransactionAmount="sum", TransactionCount="count")
yearly_summary = df.groupby(df["TransactionDate"].dt.year)["TransactionAmount"] \
    .agg(TotalTransactionAmount="sum", TransactionCount="count")

# Classify Credit / Debit / Transfer
df["TransactionCategory"] = np.select(
    [df["TransactionType"] == "Deposit",
     df["TransactionType"].isin(["Withdrawal", "Payment"]),
     df["TransactionType"] == "Transfer"],
    ["Credit", "Debit", "Transfer"], default="Other")

credit_amount = df.loc[df["TransactionCategory"] == "Credit", "TransactionAmount"].sum()
debit_amount = df.loc[df["TransactionCategory"] == "Debit", "TransactionAmount"].sum()
net_transaction_volume = credit_amount - debit_amount

# Net inflow by account
credit_by_account = df[df.TransactionCategory=="Credit"].groupby("AccountID")["TransactionAmount"].sum()
debit_by_account = df[df.TransactionCategory=="Debit"].groupby("AccountID")["TransactionAmount"].sum()
account_net_inflow = credit_by_account.subtract(debit_by_account, fill_value=0) \
    .sort_values(ascending=False)

# Dormant account detection (gap >= 60 days)
df_sorted = df.sort_values(["AccountID", "TransactionDate"]).copy()
df_sorted["PreviousTransactionDate"] = df_sorted.groupby("AccountID")["TransactionDate"].shift(1)
df_sorted["TransactionGapDays"] = (df_sorted["TransactionDate"]
                                   - df_sorted["PreviousTransactionDate"]).dt.days
dormant_transactions = df_sorted[df_sorted["TransactionGapDays"] >= 60]
dormant_accounts = dormant_transactions["AccountID"].unique()
```

Task 3 — Customer Profile Building

Rubric: activity level is based on transaction frequency — Low ≤3, Medium 4–6, High >6

64

Low Activity

customers

91

Medium Activity

customers

25

High Activity

customers

Customer Segments Identified

- High-net inflow accounts — led by ACC88286 and ACC21719
- High-frequency, low-balance accounts — e.g. ACC94907, ACC78581, ACC97411
- Negative / near-zero balance accounts — e.g. ACC78589

Task 3 — Customer Profile Building

Customer Profile with Balance & Transaction Volume:

	CustomerID	TransactionFrequency	ActivityLevel	AverageBalance	TransactionVolume
0	CUST1042	6	Medium	79417.266317	High Volume
1	CUST1114	2	Low	51474.223220	Low Volume
2	CUST1121	5	Medium	63924.640954	High Volume
3	CUST1189	3	Low	39051.942297	Low Volume
4	CUST1223	7	High	76370.221527	High Volume
5	CUST1376	5	Medium	80779.647074	High Volume
6	CUST1467	5	Medium	56689.613536	High Volume
7	CUST1497	4	Medium	90564.708385	Low Volume
8	CUST1498	8	High	65424.298341	High Volume
9	CUST1547	6	Medium	92293.667688	High Volume

Transaction Volume Counts:

```
TransactionVolume
Low Volume     97
High Volume    83
Name: count, dtype: int64
```

Final Customer Profile:

	CustomerID	TransactionFrequency	ActivityLevel	AverageBalance	TransactionVolume
0	CUST1042	6	Medium	79417.266317	High Volume
1	CUST1114	2	Low	51474.223220	Low Volume
2	CUST1121	5	Medium	63924.640954	High Volume
3	CUST1189	3	Low	39051.942297	Low Volume
4	CUST1223	7	High	76370.221527	High Volume
5	CUST1376	5	Medium	80779.647074	High Volume
6	CUST1467	5	Medium	56689.613536	High Volume
7	CUST1497	4	Medium	90564.708385	Low Volume
8	CUST1498	8	High	65424.298341	High Volume
9	CUST1547	6	Medium	92293.667688	High Volume

Average Balance by Transaction Volume:

```
TransactionVolume
High Volume    72425.479726
Low Volume     72753.311297
Name: AverageBalance, dtype: float64
```

TASK 3 — Python Code

Customer Profile Building

```
# Transaction frequency per customer
customer_profile = df.groupby("CustomerID").agg(
    TransactionFrequency=("TransactionID", "count")).reset_index()

# Activity Level rubric: Low <=3, Medium 4-6, High >6
customer_profile["ActivityLevel"] = customer_profile["TransactionFrequency"].apply(
    lambda x: "Low" if x <= 3 else ("Medium" if x <= 6 else "High"))

# Average balance per customer
average_balance = df.groupby("CustomerID")["AccountBalance"].mean()
customer_profile["AverageBalance"] = customer_profile["CustomerID"].map(average_balance)

# Transaction volume rubric: Low Volume <=4, else High Volume
customer_profile["TransactionVolume"] = customer_profile["TransactionFrequency"].apply(
    lambda x: "Low Volume" if x <= 4 else "High Volume")

final_customer_profile = customer_profile[
    ["CustomerID", "TransactionFrequency", "ActivityLevel",
     "AverageBalance", "TransactionVolume"]
].copy()

final_customer_profile.to_csv("customer_profile.csv", index=False)
```

Task 4 — Financial Risk Identification

54

Large withdrawals

≥75th percentile (Rs. 71,825)

8

Accounts w/ 2+ large withdrawals

ACC94907, ACC78581, ACC97411...

7

IQR-flagged anomalies

Outside [-Rs.23,727, Rs.1,31,813]

1

Highest-risk account

ACC94907 — all 3 risk flags

Balance volatility (std. dev.) ranges Rs. 792 – Rs. 91,165 across 184 accounts (mean ≈ Rs. 30,747). Most volatile: ACC39529, ACC46953, ACC78589 — all also flagged as high-risk.

RISK FLAG RUBRIC

1 point each for: (1) >1 large withdrawal, (2) any negative balance, (3) any near-zero balance (≤25th percentile). 20 accounts scored ≥2 out of 3 — the same accounts recur across every risk lens above.

Task 4 — Financial Risk Identification

```
Balance Volatility Statistics:
count      184.000000
mean     30746.744279
std      16582.277290
min        791.574703
25%      18377.109212
50%      29221.079624
75%      40074.992913
max      91165.113157
Name: AccountBalance, dtype: float64
```

```
Large Withdrawal Records:
    AccountID TransactionDate  TransactionAmount  AccountBalance
17    ACC70741      2023-10-16          102114.78700     68977.69984
22    ACC78581      2023-08-22           97319.12594     54321.85877
36    ACC16241      2023-08-16           76719.95995     66713.97845
69    ACC10996      2023-01-21          104027.84520     98169.32518
79    ACC72197      2024-03-23          112826.34400     42877.33441
82    ACC71938      2024-01-12           97611.73338     92037.60210
107   ACC78589      2023-09-06           77094.30183    -25971.51201
122   ACC30852      2023-03-16           91479.53628     88755.28578
123   ACC24070      2023-04-09           99268.01135     23785.60648
133   ACC19156      2024-06-11           94881.99586     81531.72277
174   ACC61827      2023-11-29           81766.58518     66464.71706
232   ACC42710      2023-07-26           80776.76325     78644.20835
244   ACC29231      2023-03-03           73175.84810     69004.46445
245   ACC77592      2023-03-10           80485.41098    112780.89430
275   ACC78581      2024-05-02           97615.08999     55054.48342
299   ACC32890      2024-03-14           77993.70895     66954.74243
305   ACC97411      2023-10-31          121570.37450     60984.40825
310   ACC81631      2023-07-26           86808.70736    122806.04430
312   ACC39161      2023-01-12           98722.86981    122005.84990
328   ACC81631      2023-11-07           81131.26780     77437.36751
```

```
Top 10 Accounts by Balance Volatility:
AccountID
ACC39529    91165.113157
ACC46953    88564.262571
ACC78589    86423.147835
ACC95164    70375.125823
ACC20297    70164.033043
ACC38559    70149.365923
ACC96868    66366.896010
ACC88252    65054.864364
ACC49140    64670.361092
ACC28305    59418.129844
Name: AccountBalance, dtype: float64
```

```
Accounts with Multiple Large Withdrawals:
AccountID
ACC11285    2
ACC24070    2
ACC94907    2
ACC97411    2
ACC77592    2
ACC81631    2
ACC71938    2
ACC78581    2
dtype: int64
```

```
IQR Anomaly Detection:
Q1: 34600.68214999999
Q3: 73485.5478875
IQR: 38884.865737500004
Lower Bound: -23726.616456250013
Upper Bound: 131812.84649375

Number of Anomalous Transactions:
7
```

TASK 4 — Python Code

Financial Risk Identification

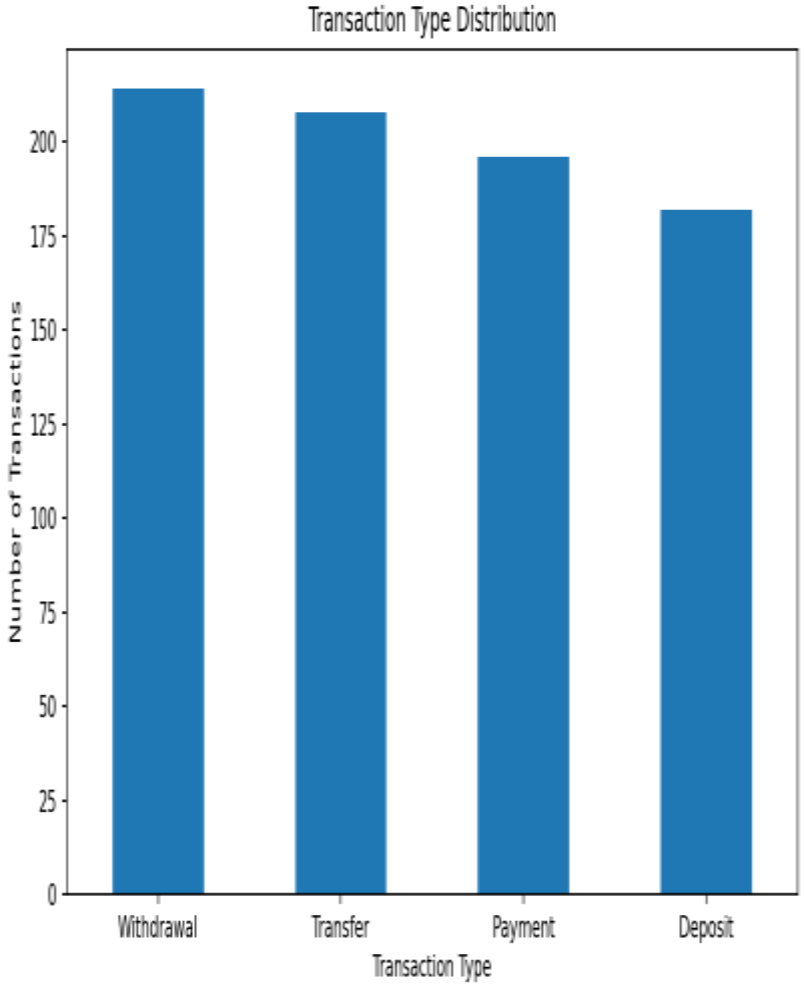
```
# Large withdrawals: >= 75th percentile of withdrawal amounts
withdrawals = df[df["TransactionType"] == "Withdrawal"]
large_withdrawal_threshold = withdrawals["TransactionAmount"].quantile(0.75)
large_withdrawals = withdrawals[withdrawals["TransactionAmount"] >= large_withdrawal_threshold]
large_withdrawal_frequency = large_withdrawals.groupby("AccountID").size() \
    .sort_values(ascending=False)

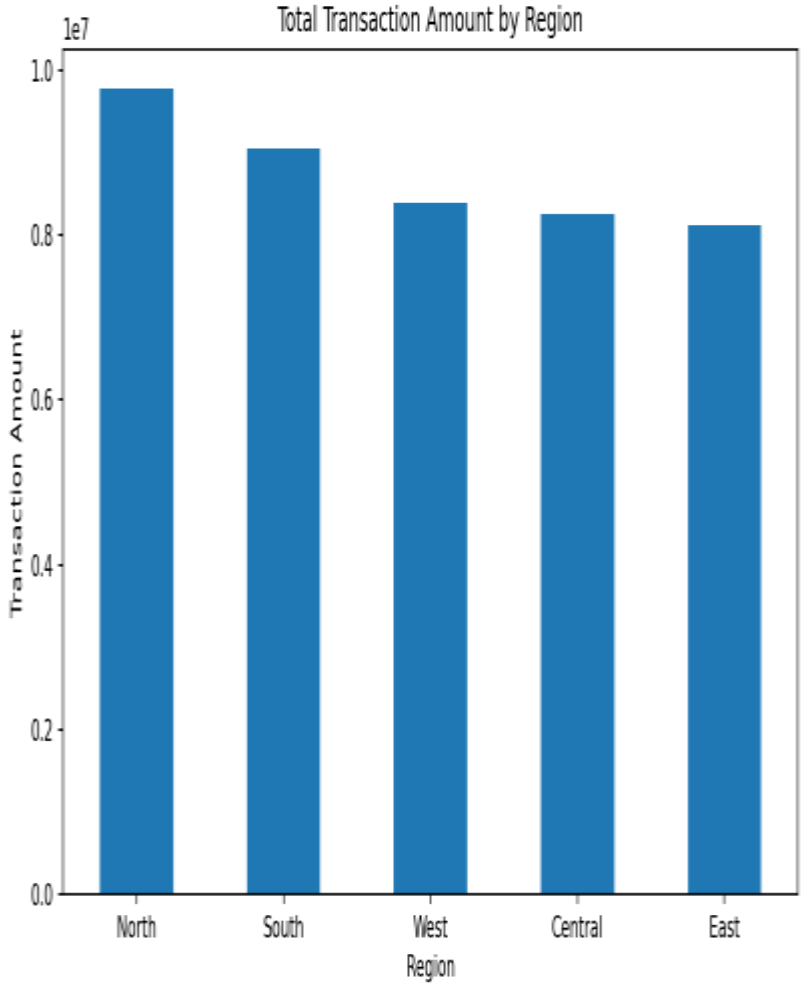
# Balance volatility (std. dev. per account)
balance_volatility = df.groupby("AccountID")["AccountBalance"].std() \
    .sort_values(ascending=False)

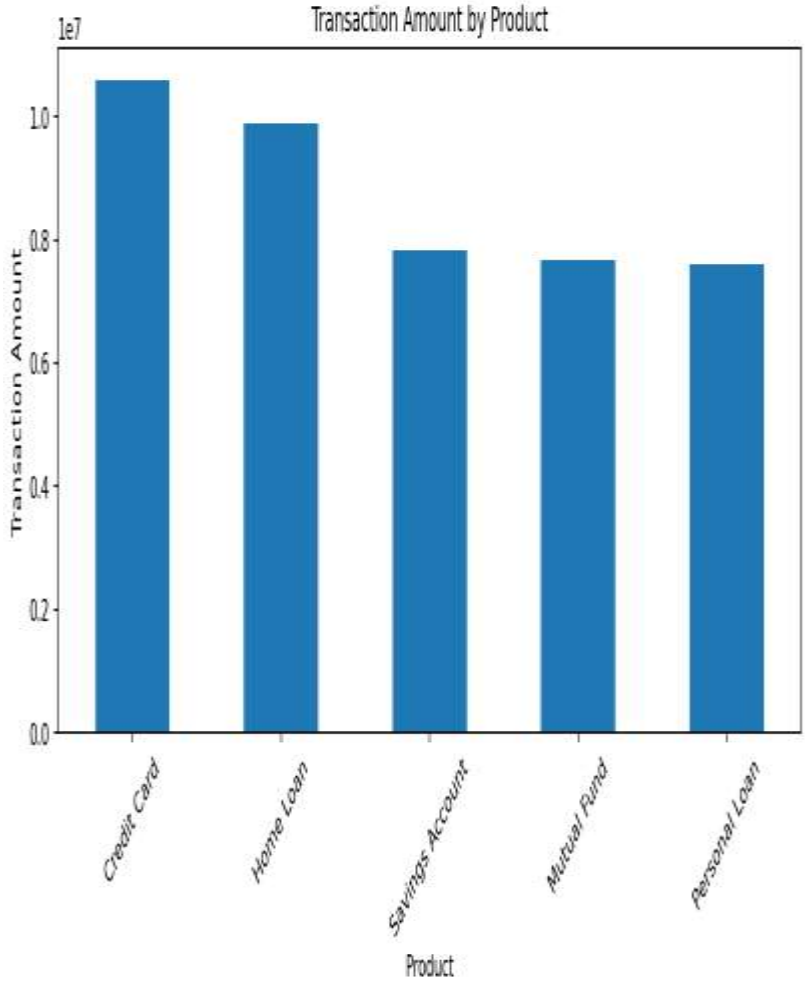
# IQR anomaly detection
Q1, Q3 = df["TransactionAmount"].quantile([0.25, 0.75])
IQR = Q3 - Q1
lower_bound, upper_bound = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
anomalies = df[(df["TransactionAmount"] < lower_bound) |
               (df["TransactionAmount"] > upper_bound)]

# Combined risk flags (1 point each, max 3)
risk_flags = pd.DataFrame(index=df["AccountID"].unique())
risk_flags["LargeWithdrawalCount"] = large_withdrawals.groupby("AccountID").size()
risk_flags["NegativeBalanceCount"] = df[df["AccountBalance"] < 0].groupby("AccountID").size()
low_balance_threshold = df["AccountBalance"].quantile(0.25)
risk_flags["NearZeroBalanceCount"] = df[df["AccountBalance"] <= low_balance_threshold] \
    .groupby("AccountID").size()
risk_flags = risk_flags.fillna(0)
risk_flags["RiskFlagCount"] = (
    (risk_flags["LargeWithdrawalCount"] > 1).astype(int) +
    (risk_flags["NegativeBalanceCount"] > 0).astype(int) +
    (risk_flags["NearZeroBalanceCount"] > 0).astype(int))
```

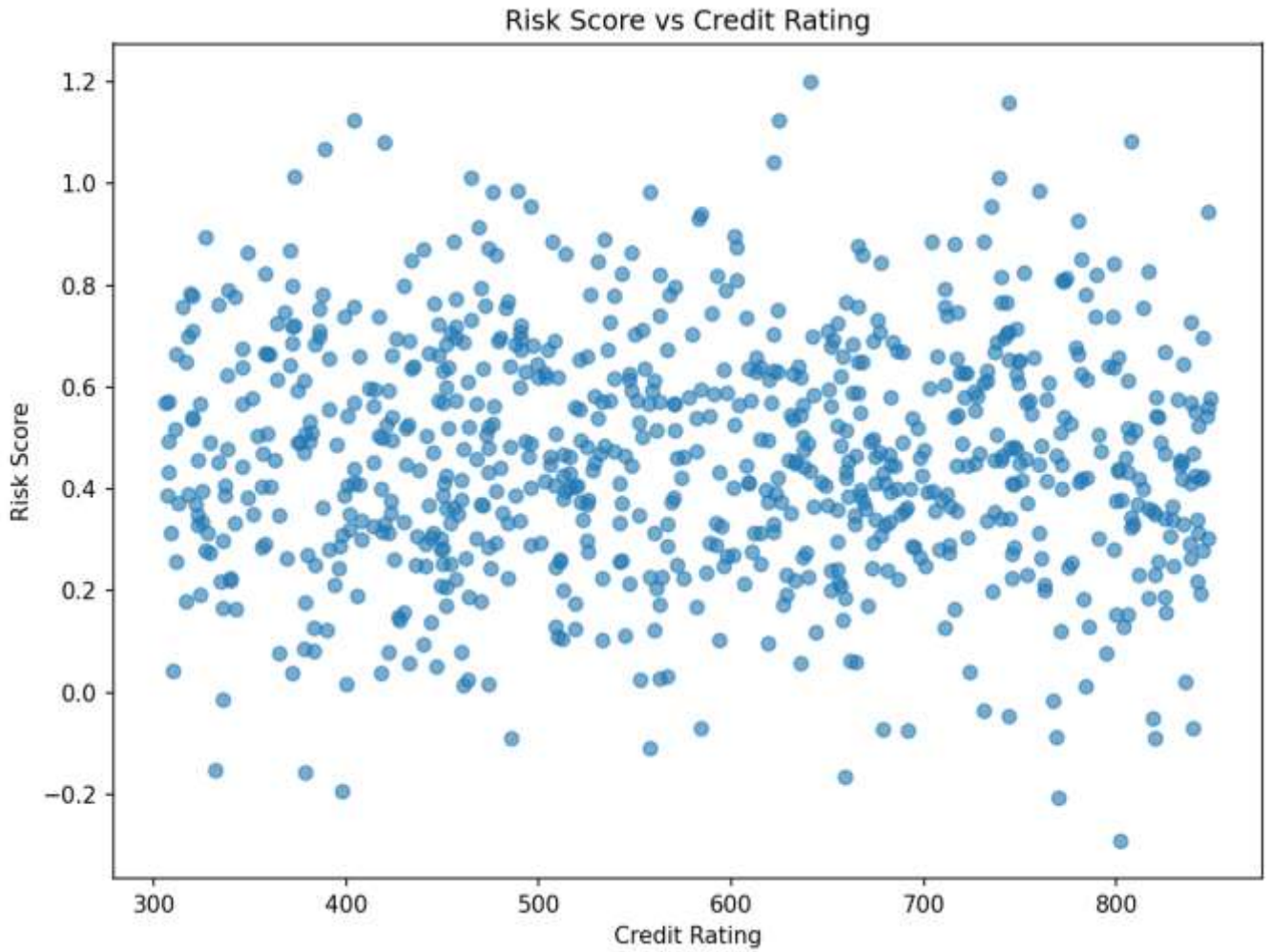
Task 5 — Visual Exploration







Task 5 — Risk Score vs. Credit Rating



OBSERVATION

No strong linear relationship is visible between CreditRating and RiskScore — points are broadly scattered at every risk level across the full rating range.

This suggests RiskScore is not simply derived from CreditRating alone, and both should be tracked independently for monitoring.

TASK 5 — Python Code

Visualisation

```
# Monthly credit vs debit trend
monthly_credit = df[df.TransactionCategory=="Credit"].groupby(
    df["TransactionDate"].dt.to_period("M"))["TransactionAmount"].sum()
monthly_debit = df[df.TransactionCategory=="Debit"].groupby(
    df["TransactionDate"].dt.to_period("M"))["TransactionAmount"].sum()

plt.plot(monthly_credit.index.astype(str), monthly_credit.values, marker="o", label="Credit")
plt.plot(monthly_debit.index.astype(str), monthly_debit.values, marker="o", label="Debit")
plt.title("Monthly Credit vs Debit Transactions"); plt.legend(); plt.show()

# Transaction type distribution
df["TransactionType"].value_counts().plot(kind="bar")

# Regional & product-wise totals
df.groupby("Region")["TransactionAmount"].sum() \
    .sort_values(ascending=False).plot(kind="bar")
df.groupby("Product")["TransactionAmount"].sum() \
    .sort_values(ascending=False).plot(kind="bar")

# Risk score vs credit rating
plt.scatter(df["CreditRating"], df["RiskScore"], alpha=0.6)
```

Task 6 — Hypothesis Testing

H_0: High-volume avg. balance ≤ Low-volume avg. balance | H_1: High-volume avg. balance > Low-volume avg. balance

Low Volume

97 customers

Rs. 72,753

avg. balance

High Volume

83 customers

Rs. 72,425

avg. balance

TEST RESULTS

T-statistic	**-0.1173**
One-tailed p-value	**0.5466**
Significance level (α)	**0.05**

Fail to reject H_0

p = 0.55 > α = 0.05 — no significant evidence high-volume accounts carry higher balances

Conclusion: transaction frequency alone is not a useful predictor of account balance — customer value should be judged by balance and inflow, not activity count.

Task 6 — Hypothesis Testing

```
Transaction Volume Groups:
VolumeGroup
Low Volume     97
High Volume    83
Name: count, dtype: int64
```

```
Average Balance by Volume Group:
VolumeGroup
High Volume    72425.479726
Low Volume     72753.311297
Name: AverageBalance, dtype: float64
```

```
Hypothesis Test Results:
T-statistic: -0.11728219779888047
One-tailed P-value: 0.5466037300295685
```

```
Hypothesis Test Conclusion:
Fail to reject H0.
There is insufficient evidence that high-volume accounts have higher average balances.
```

Combined Risk Indicators:

	LargeWithdrawalCount	NegativeBalanceCount	NearZeroBalanceCount	RiskFlagCount
ACC94907	2.0	1.0	1.0	3
ACC11062	0.0	1.0	2.0	2
ACC78581	2.0	0.0	3.0	2
ACC88252	0.0	1.0	1.0	2
ACC97411	2.0	0.0	1.0	2
ACC22036	0.0	1.0	2.0	2
ACC11285	2.0	0.0	2.0	2
ACC64022	0.0	1.0	1.0	2
ACC77592	2.0	0.0	2.0	2
ACC74631	0.0	1.0	1.0	2
ACC24070	2.0	0.0	2.0	2
ACC34821	0.0	1.0	3.0	2
ACC97225	0.0	1.0	1.0	2
ACC45101	1.0	1.0	3.0	2
ACC45521	0.0	1.0	3.0	2
ACC71938	2.0	0.0	2.0	2
ACC78589	1.0	2.0	2.0	2
ACC15671	0.0	1.0	3.0	2
ACC46953	0.0	1.0	1.0	2
ACC39529	0.0	1.0	1.0	2

TASK 6 — Python Code

Hypothesis Testing

```
from scipy.stats import ttest_ind

# Median split into High / Low volume groups
volume_threshold = customer_profile["TransactionFrequency"].median()
customer_profile["VolumeGroup"] = customer_profile["TransactionFrequency"].apply(
    lambda x: "High Volume" if x > volume_threshold else "Low Volume")

high_volume_balances = customer_profile[
    customer_profile["VolumeGroup"] == "High Volume"]["AverageBalance"]
low_volume_balances = customer_profile[
    customer_profile["VolumeGroup"] == "Low Volume"]["AverageBalance"]

# One-tailed Welch's t-test (H1: High-volume balance > Low-volume balance)
t_stat, p_value_two_tailed = ttest_ind(
    high_volume_balances, low_volume_balances, equal_var=False)

p_value = p_value_two_tailed / 2 if t_stat > 0 else 1 - (p_value_two_tailed / 2)

alpha = 0.05
if p_value < alpha:
    print("Reject H0: high-volume accounts have higher average balances.")
else:
    print("Fail to reject H0: insufficient evidence of higher balances.")
```

Key Findings

- Debits outweigh credits almost every month — net outflow of Rs. 1.17 Cr over 18 months
- 93% of accounts (172 of 184) had at least one 60+ day dormancy gap
- A recurring cluster of 7-8 accounts (incl. ACC94907, ACC78581, ACC97411) shows up across every risk lens
- 7 transactions flagged as statistical outliers via IQR — worth manual review
- Transaction frequency does not predict account balance (p = 0.55)

Recommendations

1. Run re-engagement campaigns targeting the 172 dormancy-flagged accounts
2. Set up automated monitoring for the 8 repeated large-withdrawal accounts and 20 accounts with RiskFlagCount ≥ 2
3. Manually review the 7 IQR-flagged anomalous transactions for possible fraud or data errors
4. Segment customer value by balance & net-inflow rather than transaction frequency
5. Review fee structures & set proactive liquidity alerts given the consistent debit > credit pattern

Thank You

Full analysis, cleaned datasets & code available in the accompanying report

Video Presentation Link: https://www.loom.com/share/6e7ec8f7ea974ce89e77f686d7c12a9c